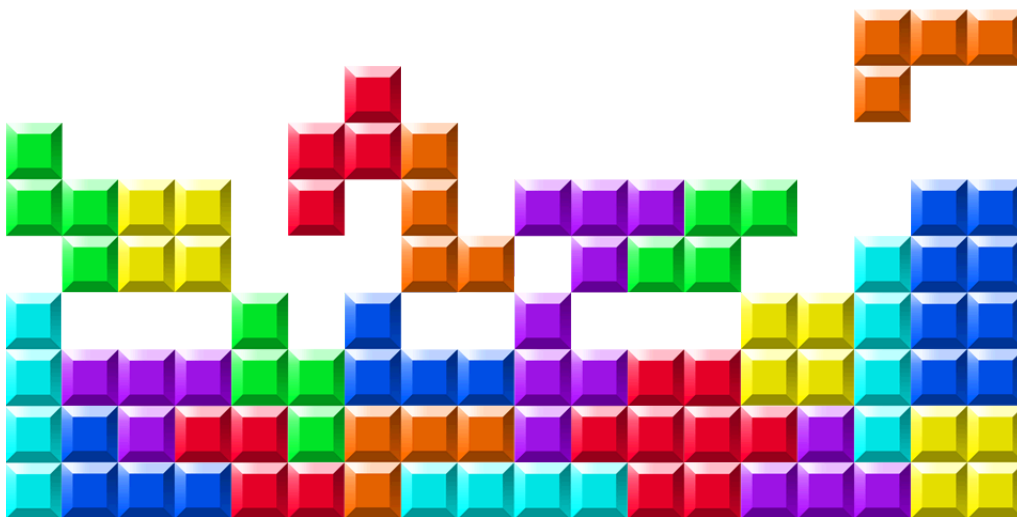

LINFO2275

Second project

Group 2:

Gavin Hope 09022500 – SINF
Jakob Thibodeau 12752500 – DATI
Lore Lernoux 77562000 – MAP



1 Introduction

2 Theoretical Background

Q-learning is a model-free reinforcement learning algorithm for learning optimal behaviour through interaction with an environment. It is model-free because it does not require a model of the environment, such as the transition probabilities or reward function. Instead, the agent learns from experience, by trying actions and observing the obtained rewards. In the standard formulation, the environment is modelled as a Markov decision process (MDP) defined by:

- a set of states $\mathcal{S}$,
- a set of actions $\mathcal{A}$,
- a transition probability distribution $p(s', r | s, a)$ that gives the probability of transitioning to state s' and receiving reward r after taking action a in state s ,
- a discount factor $\gamma \in [0, 1]$ that determines the importance of future rewards.

At time t , the agent observes a state S_t , selects an action A_t , receives a reward R_{t+1} , and moves to a new state S_{t+1} . The objective is to learn a policy that maximises the expected return, where the return is the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The discount factor γ controls how much the agent values future rewards. If γ is close to zero, the agent mainly considers immediate rewards. If γ is close to one, the agent gives more importance to long-term consequences.

The agent learns by estimating action-values, also called Q-values. A Q-value represents how good it is to take a given action in a given state. More precisely, for a policy π , the action-value function is defined as

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a],$$

which is the expected return obtained by taking action a in state s and then following policy π .

The goal of Q-learning is to learn the optimal action-value function

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a),$$

which gives the best expected return achievable from each state-action pair. Once this function is known, the optimal action in a state can be chosen greedily:

$$\pi_*(s) \in \arg \max_a q_*(s, a).$$

The optimal Q-values satisfy the Bellman optimality equation:

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

This equation expresses a recursive idea: the value of taking action a in state s is the immediate reward plus the discounted value of the best possible action in the next state. Q-learning uses this idea directly, but without needing to know the transition probabilities. Instead of computing the expectation exactly, it updates the Q-values from sampled transitions experienced by the agent [SB18].

In the tabular version of Q-learning, the agent stores one value $Q(s, a)$ for each state-action pair. After observing a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

the Q-value is updated as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

where $\alpha \in (0, 1]$ is the learning rate. It controls the size of this correction: a large value makes learning faster but less stable, while a small value makes learning slower but smoother.

A common way to select actions during learning is the ε -greedy strategy. With probability $1 - \varepsilon$, the agent chooses the action with the highest current Q-value:

$$A_t \in \arg \max_a Q(S_t, a).$$

With probability ε , it chooses a random action. This allows the agent to balance exploitation and

exploration. Exploitation means using the current knowledge to choose the best-known action, while exploration means trying other actions in order to discover potentially better strategies. In practice, ε is often decreased during training, so that the agent explores more at the beginning and exploits more once it has learned useful Q-values [Gee25].

Q-learning is an off-policy algorithm. This means that the policy used to choose actions during training does not need to be the same as the policy being learned. For example, the agent may behave according to an ε -greedy policy in order to explore, while the update itself uses the greedy action in the next state through the term

$$\max_{a'} Q(S_{t+1}, a').$$

Thus, Q-learning learns the value of the greedy policy even while following an exploratory behaviour policy. This is one of the main differences between Q-learning and on-policy methods such as SARSA [SB18].

Overall, Q-learning is a fundamental reinforcement learning algorithm because it shows how an agent can learn optimal behaviour directly from interaction.

References

- [Gee25] GeeksforGeeks. *Q-Learning in Reinforcement Learning*. Accessed: 2026-05-07. 2025. URL: <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>.
- [SB18] Richard S. Sutton and Andrew G. Barto. “Reinforcement Learning: An Introduction”. In: 2nd ed. Cambridge, MA: MIT Press, 2018.